arduino-cli

A practical learning guide for ESP8266 and ESP32 development from the terminal and VS Code, on macOS

Chapter	What you learn
1	Mental model: what arduino-cli actually does
2	Installation and configuration
3	Cores (board packages) for ESP8266 and ESP32
4	Board selection: FQBN and board options
5	Finding the serial port on macOS
6	Compile and upload
7	Serial monitor and baud rate
8	Libraries
9	sketch.yaml and profiles (reproducible projects)
10	VS Code workflow: Makefile, tasks, IntelliSense
11	Filesystem (LittleFS) uploads
12	Troubleshooting cheat sheet
13	Command quick reference and practice exercises

1. Mental model

The Arduino IDE bundles an editor, a board picker, a compiler, an uploader and a serial monitor into one window. arduino-cli is the same engine without the window. Every thing the IDE does by clicking becomes an explicit command, and that explicitness is why it feels harder at first and why it becomes more reliable later.

IDE action	CLI equivalent
Tools > Board	`-b / --fqbn` flag
Tools > Port	`-p / --port` flag
Tools > (flash size, upload speed...)	Options appended to the FQBN
Boards Manager	`arduino-cli core ...`
Library Manager	`arduino-cli lib ...`
Verify / Upload buttons	`compile` / `upload`
Serial Monitor	`arduino-cli monitor`

The four things every upload needs

- **A sketch folder** whose name equals the .ino file name.
- **A core** installed for your chip family (esp8266 or esp32).
- **A FQBN** that names the exact board and its options.
- **A port** that points to the connected board.

> **Lesson:** Most IDE problems you hit (wrong board, wrong port, wrong baud) are state problems: the IDE remembers a setting you forgot about. With the CLI the state lives in your command or in sketch.yaml, so you can read it and fix it.

2. Installation and configuration

```
brew install arduino-cli
arduino-cli version

# Create the config file (~/Library/Arduino15/arduino-cli.yaml)
arduino-cli config init

# See the effective configuration
arduino-cli config dump
```

Important folders

Path	Purpose
~/Library/Arduino15	Data directory: cores, toolchains, package indexes, config file
~/Documents/Arduino	User directory: your sketches and manually installed libraries (default)
~/Library/Arduino15/staging	Temporary download cache

> **Lesson:** The IDE and the CLI share ~/Library/Arduino15 by default. That is convenient (a core installed once works in both) but it also means the IDE running in the background can interfere with an index update or core install. Close the IDE while using the CLI.

3. Cores (board packages)

A **core** contains the compiler toolchain, the board definitions, the upload tool (esptool) and the Arduino framework for a chip family. Without it, the CLI has no idea what an ESP8266 is.

Add package index URLs

```
arduino-cli config add board_manager.additional_urls \
  http://arduino.esp8266.com/stable/package_esp8266com_index.json

arduino-cli config add board_manager.additional_urls \
  https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

arduino-cli core update-index
```

Install and manage cores

```
arduino-cli core search esp8266
arduino-cli core install esp8266:esp8266
arduino-cli core install esp32:esp32
arduino-cli core install esp8266:esp8266@3.1.2   # pin a specific version

arduino-cli core list              # installed cores
arduino-cli core list --all
arduino-cli core upgrade           # upgrade everything
arduino-cli core uninstall esp8266:esp8266
```

Lesson: Always run `core update-index` after adding a URL, otherwise the core cannot be found. The esp32 core is large (hundreds of MB) so the first install takes time on a slow connection. Retry if a download is interrupted.

Good practice: Upgrading a core can change behaviour of your existing code. For commercial products, pin versions (see Chapter 9) and upgrade deliberately.

4. Board selection: FQBN and options

FQBN means **Fully Qualified Board Name**. It has the form `vendor:architecture:board`, optionally followed by `:option=value,option=value`.

```
esp8266:esp8266:nodemcuv2
 |        |       |
 |        |       +-- board id
 |        +---------- architecture
 +------------------- vendor / package
```

Finding the right FQBN

```
arduino-cli board listall                # all boards of installed cores
arduino-cli board listall esp8266        # filter by text
arduino-cli board listall nodemcu
arduino-cli board list                   # connected boards and ports
```

Common FQBNs

Board	FQBN
NodeMCU 1.0 (ESP-12E)	esp8266:esp8266:nodemcuv2
Wemos / LOLIN D1 Mini	esp8266:esp8266:d1_mini
Generic ESP8266 module	esp8266:esp8266:generic
ESP32 Dev Module	esp32:esp32:esp32
ESP32-S3 Dev Module	esp32:esp32:esp32s3
ESP32-C3 Dev Module	esp32:esp32:esp32c3

> **Lesson:** `board list` often prints Unknown for ESP boards. This is normal: the USB-to-serial chip (CP2102, CH340) cannot tell the computer which board is behind it. You always supply the FQBN yourself.

Board options

Everything under the IDE's Tools menu (flash size, upload speed, partition scheme, CPU frequency) is a board option. List them with `board details`:

```
arduino-cli board details -b esp8266:esp8266:nodemcuv2

# Use options by appending them to the FQBN
arduino-cli compile -b esp8266:esp8266:nodemcuv2:baud=921600,eesz=4M2M,xtal=80 .

# ESP32 example
arduino-cli compile -b esp32:esp32:esp32:PartitionScheme=min_spiffs,UploadSpeed=460800 .
```

> **Lesson:** A wrong flash size or partition scheme is a classic cause of 'uploads fine but crashes or never boots'. If you use LittleFS, the flash split (for example eesz=4M2M, meaning 4 MB flash with 2 MB filesystem) must be the same when you build the firmware and when you build the filesystem image.

5. Finding the serial port on macOS

```
ls /dev/cu.*
arduino-cli board list
```

Typical name	Usually means
/dev/cu.usbserial-0001, /dev/cu.usbserial-1410	CP2102 or CH340 style chip
/dev/cu.SLAB_USBtoUART	Silicon Labs CP210x with vendor driver
/dev/cu.wchusbserial*	WCH CH340/CH341 with vendor driver
/dev/cu.usbmodem*	Native USB (ESP32-S2/S3/C3 in USB-CDC mode)

- Use `/dev/cu.*` for outgoing connections. The matching `/dev/tty.*` entry waits for a carrier signal and can hang.
- The name **changes** if you move the cable to another USB port or hub. Re-check after every reconnect.
- **No port at all?** First suspect a charge-only USB cable. Try another cable before anything else. Then check the board's USB chip driver.
- A port that appears and disappears repeatedly usually means weak power or a bad cable.

6. Compile and upload

```
# Compile only (checks the code, produces a binary)
arduino-cli compile -b esp8266:esp8266:nodemcuv2 .

# Upload an already compiled sketch
arduino-cli upload -b esp8266:esp8266:nodemcuv2 -p /dev/cu.usbserial-0001 .

# Compile and upload in one step
arduino-cli compile --upload -b esp8266:esp8266:nodemcuv2 -p /dev/cu.usbserial-0001 .

# Useful flags
--verbose          # print every command executed (best debugging tool)
--warnings all     # show all compiler warnings
--clean            # ignore build cache
--build-path ./build   # keep build output in a known place
--export-binaries  # copy .bin files to the sketch's build folder
```

Lesson: The sketch folder name must match the .ino file: `blink/blink.ino`. The trailing dot in the commands means 'the sketch in the current folder'. If you run the command from another folder, give the path instead.

Typical project layout

```
smart-plug/
  smart-plug.ino      <- main sketch, same name as the folder
  config.h            <- your own header files live beside it
  sketch.yaml         <- board + port defaults (Chapter 9)
  data/               <- files for LittleFS (Chapter 11)
```

What happens during an ESP upload

- The CLI invokes **esptool** (bundled in the core) over the serial port.
- esptool toggles the DTR and RTS lines to put the chip in bootloader mode. Most dev boards wire this automatically.
- On bare modules or poorly designed boards you may have to hold BOOT/FLASH (GPIO0 low) while resetting.
- The firmware is written at the chosen upload speed. Lower it if the connection is unstable.

7. Serial monitor and baud rate

```
arduino-cli monitor -p /dev/cu.usbserial-0001 -c baudrate=115200

# Alternatives
screen /dev/cu.usbserial-0001 115200     # exit: Ctrl+A then K then Y
```

Rules that remove most serial problems

- **Baud must match the code.** `Serial.begin(115200)` requires the monitor at 115200.
- **Garbage characters** almost always mean a baud mismatch. The ESP8266 ROM bootloader prints at 74880 baud, so a short burst of garbage right at reset is normal.
- **Only one program may hold the port.** A running monitor, the Arduino IDE monitor, or screen will block uploads with 'port busy'.
- **Exit the CLI monitor with Ctrl+C.** Always close it before uploading.
- Opening the port often resets the board. If output seems to start late, wait a second or press reset.
- Print a clear boot banner (firmware version, reset reason) so you can confirm you are looking at the right firmware.

```
void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("Boot: smart-plug v1.0.3");
}
```

> **Do not confuse:** Upload speed and monitor baud are two different numbers. Upload speed (for example 921600) is only for flashing. Monitor baud must equal what you pass to Serial.begin().

8. Libraries

```
arduino-cli lib search "WiFiManager"
arduino-cli lib install "WiFiManager"
arduino-cli lib install "PubSubClient@2.8.0"
arduino-cli lib list
arduino-cli lib upgrade
arduino-cli lib uninstall "WiFiManager"

# Install from a Git repo or local zip (needs unsafe install enabled)
arduino-cli config set library.enable_unsafe_install true
arduino-cli lib install --git-url https://github.com/user/repo.git
arduino-cli lib install --zip-path ./MyLib.zip
```

- Library names are exact. Run `lib search` first and copy the name.
- The library name and the header you `#include` can differ.
- Libraries for a specific architecture (esp8266 or esp32) are only compiled when that architecture is selected.
- If two libraries provide the same header, the compiler may pick the wrong one. Use `--verbose` to see which path was used.
- For commercial firmware, pin library versions so a new release cannot silently change behaviour.

9. sketch.yaml and profiles

Typing the FQBN and port every time is error prone. **sketch.yaml** stores them next to the sketch.

```
arduino-cli board attach -p /dev/cu.usbserial-0001 -b esp8266:esp8266:nodemcuv2

# afterwards, flags are optional
arduino-cli compile --upload .
```

Profiles: reproducible builds

A profile pins the board, the core version and the library versions. Anyone (or you, in 6 months) gets the same build.

```
profiles:
  default:
    fqbn: esp8266:esp8266:nodemcuv2
    platforms:
      - platform: esp8266:esp8266 (3.1.2)
        platform_index_url: http://arduino.esp8266.com/stable/package_esp8266com_index.json
    libraries:
      - WiFiManager (2.0.17)
      - PubSubClient (2.8)
default_profile: default
default_port: /dev/cu.usbserial-0001

arduino-cli compile --profile default .
arduino-cli upload  --profile default -p /dev/cu.usbserial-0001 .
```

Why it matters: When a profile is used, the CLI downloads the exact versions into a separate folder, so your global installs are not touched. This is the best defence against 'it worked last month' problems.

10. VS Code workflow

Option A: Makefile

```
FQBN = esp8266:esp8266:nodemcuv2:baud=921600,eesz=4M2M
PORT = $(shell ls /dev/cu.usbserial* 2>/dev/null | head -1)

build:
	arduino-cli compile -b $(FQBN) .

upload:
	arduino-cli compile --upload -b $(FQBN) -p $(PORT) .

monitor:
	arduino-cli monitor -p $(PORT) -c baudrate=115200

clean:
	rm -rf build
```

Run `make upload` then `make monitor`. Makefiles require real tab characters before recipe lines.

Option B: VS Code tasks (.vscode/tasks.json)

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Upload",
      "type": "shell",
      "command": "arduino-cli compile --upload .",
      "group": { "kind": "build", "isDefault": true }
    },
    {
      "label": "Monitor",
      "type": "shell",
      "command": "arduino-cli monitor -c baudrate=115200",
      "isBackground": true
    }
  ]
}
```

With sketch.yaml in place the commands stay this short. Press Cmd+Shift+B to run the default build task.

IntelliSense (autocomplete and error squiggles)

```
arduino-cli compile -b esp8266:esp8266:nodemcuv2 --only-compilation-database .
# produces build/compile_commands.json for clangd or the C/C++ extension
```

- Install the **clangd** extension (or Microsoft C/C++) and point it at the generated compile_commands.json.
- Regenerate it whenever you add libraries or change the board.
- Keep your sketch as .ino for simplicity, or move logic into .h/.cpp files to get better editor support as the project grows.

11. Filesystem (LittleFS) uploads

arduino-cli does **not** have a built-in command that uploads the sketch's data folder, which the IDE does through a plugin. You build a filesystem image and flash it yourself.

Concept

- Put files in `data/` inside the sketch folder.
- Create an image with **mklittlefs**, using the same size as the partition for your chosen flash layout.
- Flash the image with **esptool** at the filesystem's start address.
- Both tools ship inside the esp8266 core under ~/Library/Arduino15/packages/esp8266/tools/.

```
# Find the tools
ls ~/Library/Arduino15/packages/esp8266/tools/

# Sketch of the flow (sizes/addresses depend on your flash layout):
mklittlefs -c data -p 256 -b 8192 -s <fs_size_bytes> littlefs.bin
esptool.py --port /dev/cu.usbserial-0001 write_flash <fs_start_address> littlefs.bin
```

Careful: The address and size come from the linker script of your chosen flash layout (for example eesz=4M2M). Look in the core's `tools/sdk/ld/` folder, or run a verbose compile and read the generated values. If the numbers are wrong, the filesystem will fail to mount. Practise on a spare board first.

- Alternative: LittleFS can also be created at runtime from code (write files with the LittleFS API) so you do not need an image at all.
- Alternative: use PlatformIO, which has a first-class 'Upload Filesystem Image' command, if this step becomes a daily need.

12. Troubleshooting cheat sheet

Symptom	Likely cause and fix
Port not listed	Charge-only cable; try another cable and USB port. Check driver for CH340 on older macOS.
Failed to connect / No serial data received	Wrong port, board not in bootloader mode. Hold BOOT/FLASH (GPIO0 low) while starting upload; check wiring on bare modules.
Resource busy / port busy	Serial monitor (CLI, IDE or screen) still open. Close it, then upload.
Permission denied	Stale port name after reconnect. Run ls /dev/cu.* again.
Platform not found / unknown board	Missing core or index. Run core update-index, core install, check the FQBN spelling.
Main file missing from sketch	Folder name differs from .ino name.
Library header not found	Library not installed; the library name differs from the header name. Use lib search.
Compiles but device reboots in a loop	Wrong flash size/partition option, brown-out from weak power, or a crash in setup. Read the serial output.
Garbage on serial monitor	Baud mismatch (except the normal 74880 boot burst on ESP8266).
Upload fails halfway	Lower the upload speed (e.g. 460800 or 115200); use a shorter cable.
Everything worked, now it does not	A core or library upgraded. Use a profile to pin versions.
Strange behaviour after moving to another Mac	Different cores/libraries installed. Use profiles; run core list and lib list to compare.

A debugging routine that works

- 1. Run the same command with `--verbose` and read the first error, not the last.
- 2. Confirm the port: `ls /dev/cu.*`.
- 3. Confirm the FQBN: `arduino-cli board listall esp8266`.
- 4. Close every program that might hold the port.
- 5. Try a different cable, then a different USB port.
- 6. Compile a minimal Blink sketch for the same board. If this works, the problem is in your code or libraries, not the toolchain.
- 7. Check `arduino-cli config dump` and `core list` for surprises.

13. Quick reference

Task	Command
Update indexes	arduino-cli core update-index
Install core	arduino-cli core install esp8266:esp8266
List boards	arduino-cli board listall esp8266
Board options	arduino-cli board details -b
List ports	ls /dev/cu.*
Compile	arduino-cli compile -b .

Task	Command
Upload	arduino-cli upload -b -p .
Compile + upload	arduino-cli compile --upload -b -p .
Monitor	arduino-cli monitor -p -c baudrate=115200
Search library	arduino-cli lib search
Install library	arduino-cli lib install ""
Save defaults	arduino-cli board attach -p -b
Show config	arduino-cli config dump
Verbose build	arduino-cli compile --verbose ...
Help for any command	arduino-cli --help

Practice exercises

- **Exercise 1.** Install the esp8266 core, find your board's FQBN with board listall, and compile an empty sketch.
- **Exercise 2.** Upload Blink. Change the delay, upload again, and confirm the change.
- **Exercise 3.** Print a boot banner at 115200 and read it with arduino-cli monitor. Then deliberately set the wrong baud and observe the garbage.
- **Exercise 4.** Run the monitor and try to upload at the same time. Read the error. Fix it.
- **Exercise 5.** Create sketch.yaml with board attach and build with just `arduino-cli compile --upload .`
- **Exercise 6.** Install a library (for example WiFiManager), use it in a sketch, then pin its version in a profile.
- **Exercise 7.** Write a Makefile or VS Code task so that one shortcut compiles, uploads and opens the monitor.
- **Exercise 8.** Run compile with --verbose and find where the compiler, linker and esptool commands appear in the output.
- **Exercise 9 (advanced).** Change the board option to a different flash split, see how the binary size report changes, then try the LittleFS flow on a spare board.

> **Study plan:** Learn it in this order: Chapters 1 to 7 give you a working daily workflow. Chapters 8 to 10 make it comfortable. Chapters 9 and 11 make it professional and reproducible, which matters most once you ship products to customers.